

AI-GENIE de novos itens

Frederico Pedrosa

2026-07-26

Contents

1	Chamar os dados	1
2	Transformação em embeddings	2
3	Protocolo AI-GENIE	2
3.1	Definição do estimador	2
3.2	Loop AI-GENIE	2
3.3	Relatório de itens excluídos	7
4	EGA com itens escolhidos e NMI final	8
4.1	Selecionar itens teoricamente válidos	10
4.2	Salvar matriz de itens estáveis e não redundantes	12

1 Chamar os dados

```
library(readxl)
url <- "https://raw.githubusercontent.com/FredPedrosa/Generative_Psychometrics/main/template/itens/itens.xlsx"

# Download
arquivo_temp <- tempfile(fileext = ".xlsx")
download.file(url, destfile = arquivo_temp, mode = "wb")
dados <- read_excel(arquivo_temp)

# Visualização os dados
str(dados)

## tibble [90 x 2] (S3: tbl_df/tbl/data.frame)
## $ item : chr [1:90] "i01" "i02" "i03" "i04" ...
## $ texto: chr [1:90] "Tenho dificuldade para sentir prazer nas atividades cotidianas" "Durante situaçã
```

```
dados <- as.data.frame(dados)
```

2 Transformação em embeddings

```
# vetor de embeddings
vetor_de_textos <- dados$texto
matriz_embeddings <- er_embed(vetor_de_textos, api = "cohere", model = "embed-multilingual-v3.0")
rownames(matriz_embeddings) <- dados$item

# Tratamento de matriz para AI-GENIE em matriz numérica
matriz <- t(matriz_embeddings)
head(matriz[, 1:5])
```

```
##           i01           i02           i03           i04           i05
## [1,] -0.0339050300 -0.028823853 -0.028366089 0.0066986084 0.007740021
## [2,] -0.0039215090 -0.001775742 -0.004127502 0.0009264946 0.043823242
## [3,] 0.0088043210 0.050720215 0.056488037 0.0143508910 0.044952393
## [4,] 0.0000641346 -0.010292053 -0.007419586 0.0040054320 0.023071289
## [5,] 0.0073051453 -0.024246216 -0.023818970 0.0131912230 -0.026138306
## [6,] -0.0001929998 0.003978729 -0.005870819 0.0591125500 -0.009254456
```

```
dim(matriz)
```

```
## [1] 1024 90
```

3 Protocolo AI-GENIE

3.1 Definição do estimador

Glasso teve melhor performance

```
# 1. Criar o gabarito teórico exato para os 90 itens
gabarito_teorico <- c(rep("Depressao", 30), rep("Ansiedade", 30),
                     rep("Estresse", 30))

# Amarrar os nomes das colunas (i01 a i90) ao gabarito
names(gabarito_teorico) <- colnames(matriz)

# Visualizar o gabarito
head(gabarito_teorico)
```

```
##           i01           i02           i03           i04           i05           i06
## "Depressao" "Depressao" "Depressao" "Depressao" "Depressao" "Depressao"
```

3.2 Loop AI-GENIE

```
library(EGAnet)
# 1. Definir os dados iniciais e parâmetros
dados_iter <- matriz
```

```

stability_threshold <- 0.75
iteracao <- 1

# CRIANDO A LISTA PARA O RELATÓRIO
relatorio_iteracoes <- list()

# 2. Iniciar o Loop Automático
repeat {
  cat("\n=====\\n")
  cat(" INICIANDO ITERAÇÃO", iteracao, "| Total de variáveis:", ncol(dados_iter), "\\n")
  cat("=====\\n")

  # Salvamos os nomes dos itens
  nomes_iniciais <- colnames(dados_iter)

  # -----
  # ETAPA 1: UVA (Análise de Redundância)
  # -----
  cat("[1/2] Rodando UVA...\\n")
  uva_res <- UVA(data = dados_iter)

  if (!is.null(uva_res$redundant) && length(uva_res$redundant) > 0) {
    dados_apos_uva <- uva_res$reduced_data
  } else {
    dados_apos_uva <- dados_iter
  }

  # Verificando exatamente quem sumiu e quem apareceu no UVA
  nomes_pos_uva <- colnames(dados_apos_uva)
  itens_excluidos_uva <- setdiff(nomes_iniciais, nomes_pos_uva) # Quem saiu
  itens_novos_uva <- setdiff(nomes_pos_uva, nomes_iniciais)      # Quem foi criado

  # -----
  # ETAPA 2: bootEGA (Checagem de Estabilidade)
  # -----
  cat("[2/2] Rodando bootEGA (limite ", stability_threshold, ")...\\n", sep="")
  boot_atual <- bootEGA(
    data = dados_apos_uva,
    iter = 500,
    type = "parametric",
    model = "glasso",
    seed = 123,
    plot.typicalStructure = FALSE
  )

  stabilities <- boot_atual$stability$item.stability$item.stability$empirical.dimensions
  itens_estaveis <- names(stabilities[stabilities >= stability_threshold])
  itens_instaveis <- names(stabilities[stabilities < stability_threshold])

  # Atualiza o banco mantendo só os estáveis para a próxima rodada
  dados_iter <- dados_apos_uva[, itens_estaveis]

  # -----

```

```

# ETAPA 3: SALVAR RELATÓRIO DA ITERAÇÃO ATUAL
# -----
# Guardamos tudo numa lista estruturada
relatorio_iteracoes[[paste0("Iteracao_", iteracao)]] <- list(
  Qtd_Itens_Iniciais = length(nomes_iniciais),
  UVA_Itens_Modificados_Removidos = itens_excluidos_uva,
  UVA_Novos_Itens_Criados = itens_novos_uva,
  bootEGA_Itens_Instaveis_Removidos = itens_instaveis,
  Qtd_Itens_Finais = ncol(dados_iter)
)

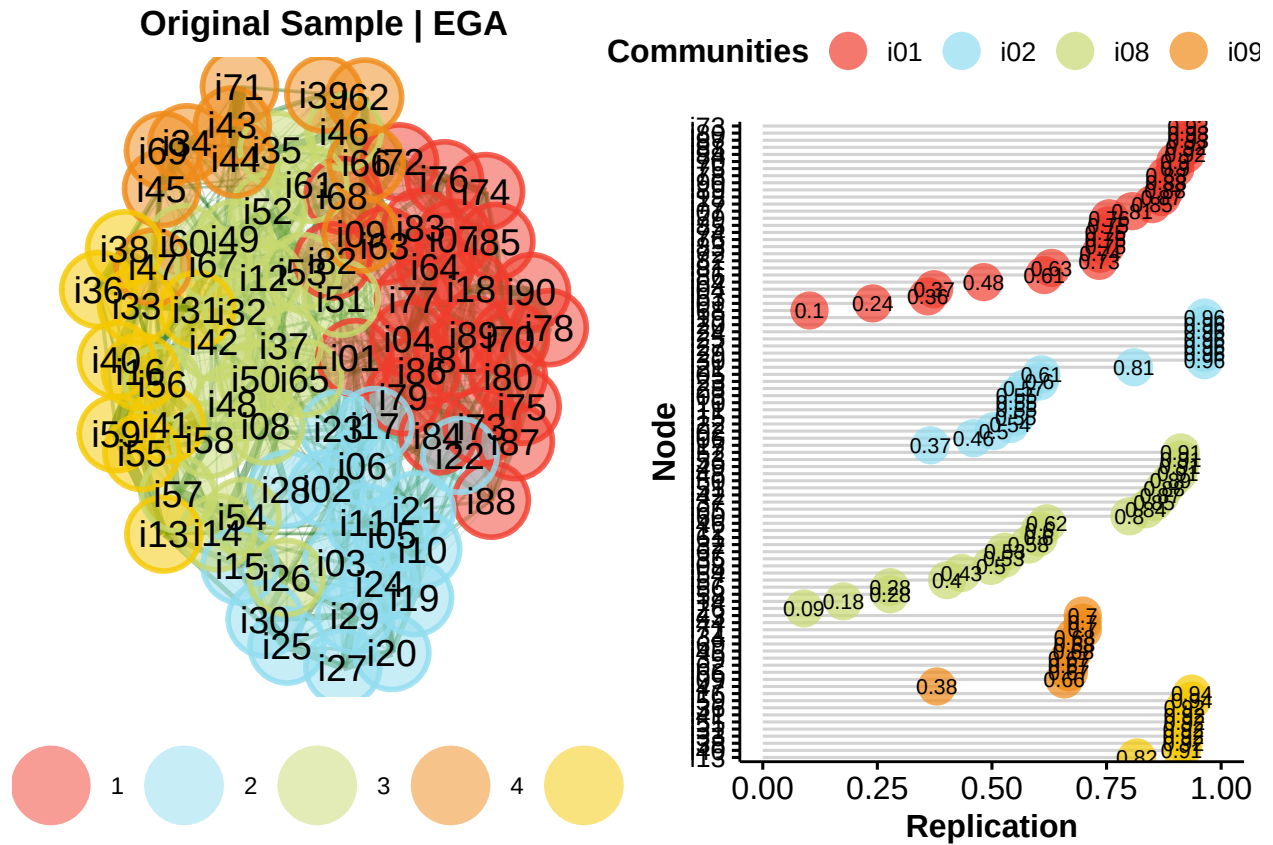
# -----
# ETAPA 4: CONDIÇÃO DE PARADA
# -----
if (length(itens_excluidos_uva) == 0 && length(itens_instaveis) == 0) {
  cat("\n SUCESSO ABSOLUTO! Purificação concluída.\n")
  break
}

iteracao <- iteracao + 1
}

##
## =====
## INICIANDO ITERAÇÃO 1 | Total de variáveis: 90
## =====
## [1/2] Rodando UVA...
## [2/2] Rodando bootEGA (limite 0.75)...

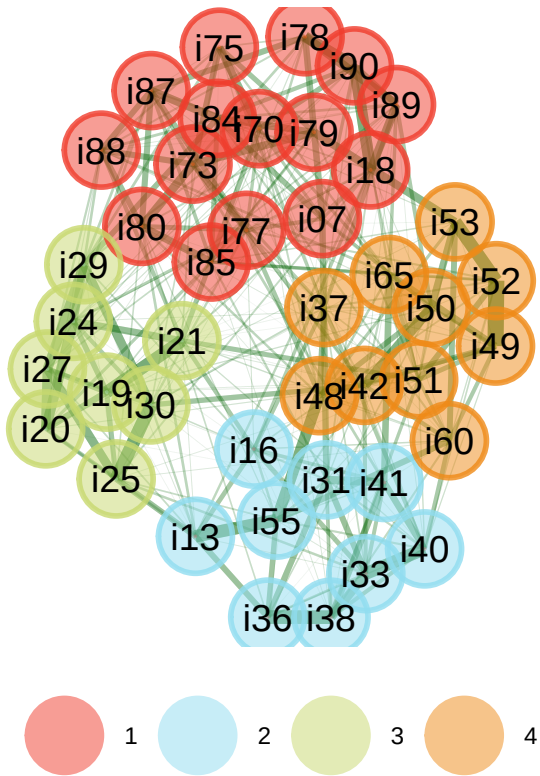
##
## =====
## INICIANDO ITERAÇÃO 2 | Total de variáveis: 43
## =====
## [1/2] Rodando UVA...
## [2/2] Rodando bootEGA (limite 0.75)...

```

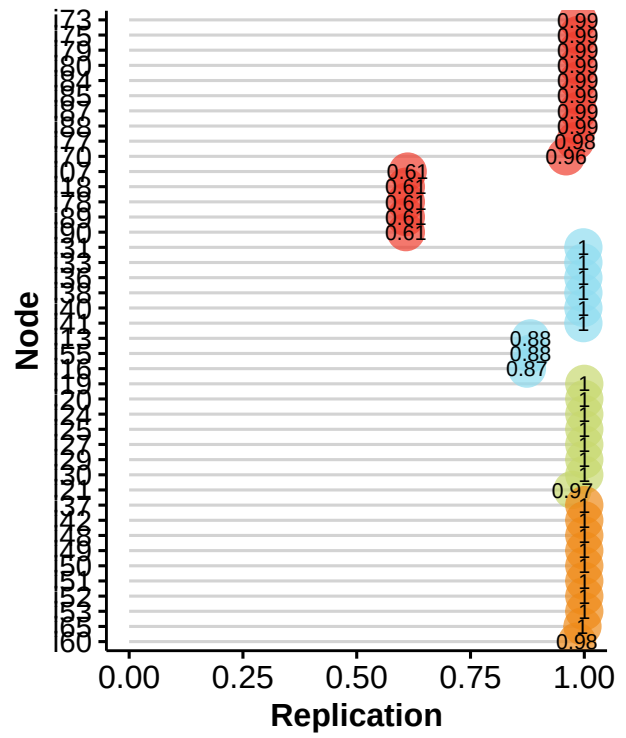


```
##
## =====
## INICIANDO ITERAÇÃO 3 | Total de variáveis: 37
## =====
## [1/2] Rodando UVA...
## [2/2] Rodando bootEGA (limite 0.75)...
```

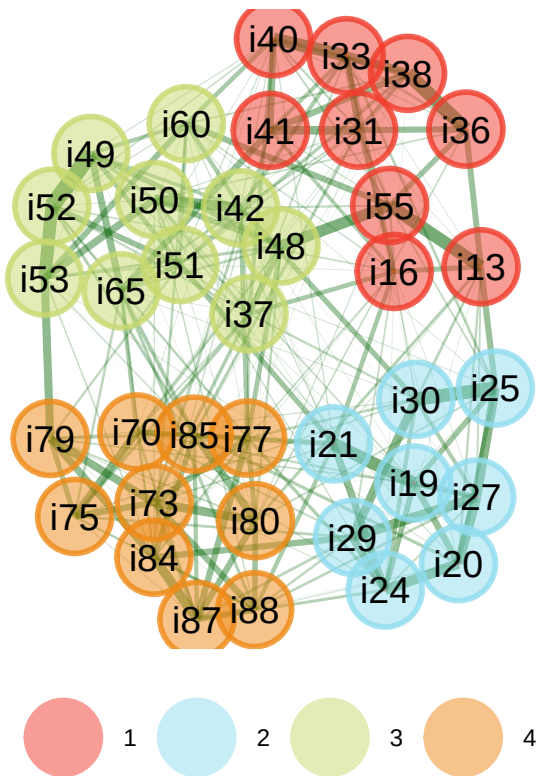
Original Sample | EGA



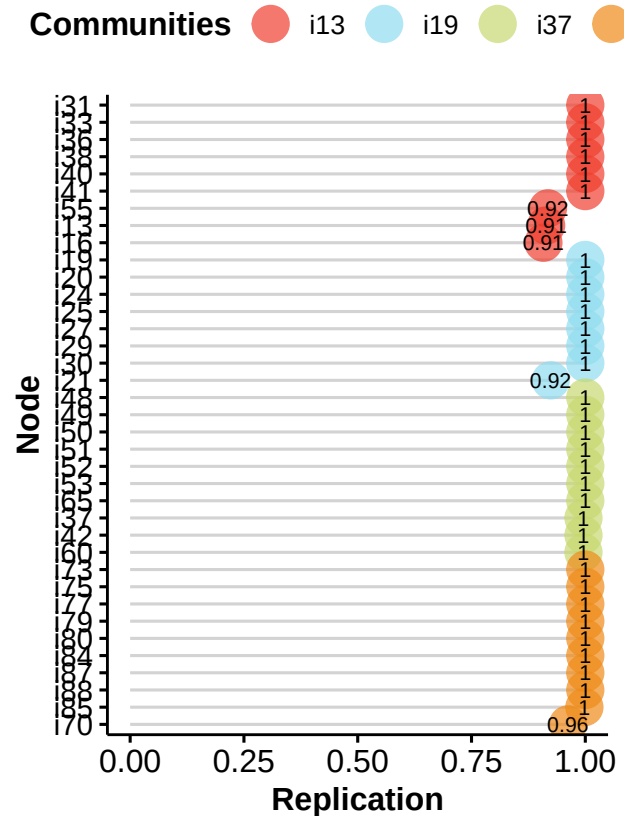
Communities i07 i13 i19



Original Sample | EGA



Communities



```
##
##  SUCESSO ABSOLUTO! Purificação concluída.

# =====
# 3. RESULTADOS FINAIS
# =====

# AQUI ESTÁ O ÚLTIMO OBJETO SELECIONADO
matriz_final_purificada <- dados_iter

cat("\nA MATRIZ FINAL FOI SALVA NO OBJETO 'matriz_final_purificada' com", ncol(matriz_final_purificada))

##
## A MATRIZ FINAL FOI SALVA NO OBJETO 'matriz_final_purificada' com 37 itens.
```

3.3 Relatório de itens excluídos

```
for (nome_iter in names(relatorio_iteracoes)) {
  cat("\n---", toupper(nome_iter), "---\n")
  cat("Iniciou com:", relatorio_iteracoes[nome_iter]$Qtd_Itens_Iniciais, "itens\n")

  if(length(relatorio_iteracoes[nome_iter]$UVA_Itens_Modificados_Removidos) > 0) {
    cat("UVA removeu/juntou:", paste(relatorio_iteracoes[nome_iter]$UVA_Itens_Modificados_Removidos, "
```

```

}
if(length(relatorio_iteracoes[[nome_iter]]$bootEGA_Itens_Instaveis_Removidos) > 0) {
  cat("bootEGA excluiu (<0.75):", paste(relatorio_iteracoes[[nome_iter]]$bootEGA_Itens_Instaveis_Removidos, collapse=" "))
}

cat("Terminou com:", relatorio_iteracoes[[nome_iter]]$Qtd_Itens_Finais, "itens\n")
}

```

```

##
## --- ITERACAO_1 ---
## Iniciou com: 90 itens
## bootEGA excluiu (<0.75): i01, i02, i03, i04, i05, i06, i08, i09, i10, i11, i12, i14, i15, i17, i22, i23, i24, i25, i26, i27, i28, i29, i30, i31, i32, i33, i34, i35, i36, i37, i38, i39, i40, i41, i42, i43, i44, i45, i46, i47, i48, i49, i50, i51, i52, i53, i54, i55, i56, i57, i58, i59, i60, i61, i62, i63, i64, i65, i66, i67, i68, i69, i70, i71, i72, i73, i74, i75, i76, i77, i78, i79, i80, i81, i82, i83, i84, i85, i86, i87, i88, i89, i90, i91, i92, i93, i94, i95, i96, i97, i98, i99, i100
## Terminou com: 43 itens
##
## --- ITERACAO_2 ---
## Iniciou com: 43 itens
## UVA removeu/juntou: i59
## bootEGA excluiu (<0.75): i07, i18, i78, i89, i90
## Terminou com: 37 itens
##
## --- ITERACAO_3 ---
## Iniciou com: 37 itens
## Terminou com: 37 itens

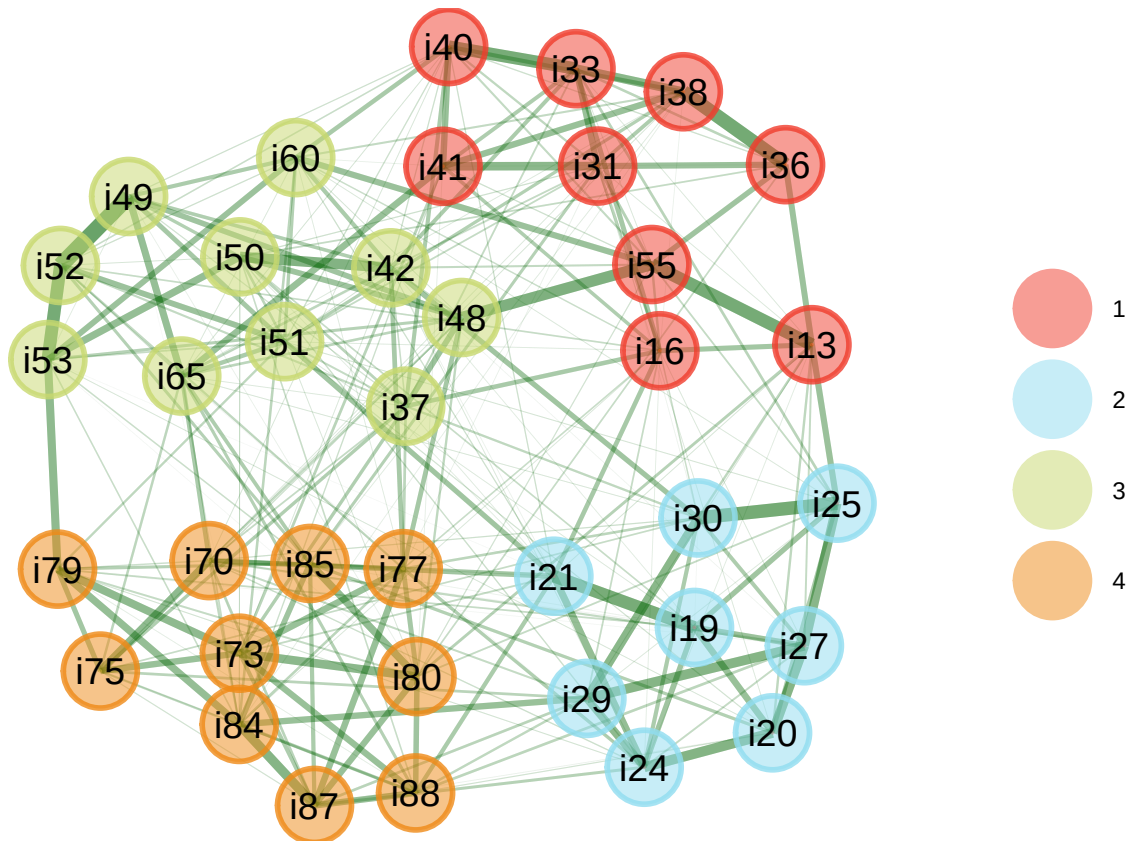
```

4 EGA com itens escolhidos e NMI final

```

# =====
# 1. RODAR EGA COM OS 39 ITENS SELECIONADOS
# =====
set.seed(123)
ega_final <- EGA(
  data = matriz_final_purificada,
  model = "glasso",
  plot.EGA = TRUE
)

```

```
# =====
# 2. ADAPTAR O GABARITO (DE 90 PARA 39)
# =====
# Extrai os nomes das 39 colunas que sobreviveram
itens_sobreviventes <- colnames(matriz_final_purificada)

# Cria um vetor vazio para guardar o novo gabarito
gabarito_purificado <- c()

# Loop para alinhar cada item sobrevivente com sua dimensão teórica
for (nome_item in itens_sobreviventes) {

  # Se o nome original do item (ex: "i13") existir no gabarito...
  if (nome_item %in% names(gabarito_teorico)) {
    gabarito_purificado[nome_item] <- gabarito_teorico[nome_item]
  } else {
    item_base <- strsplit(nome_item, "_")[[1]][1]
    gabarito_purificado[nome_item] <- gabarito_teorico[item_base]
  }
}

# =====
# 3. CALCULAR O NOVO NMI
# =====
comparacao_purificada <- community.compare(
```

```

base = gabarito_purificado,
comparison = ega_final$wc,
method = "nmi"
)

# Visualizar o resultado
cat("\nNMI ORIGINAL (com 90 itens): 0.4511239\n")

```

```

##
## NMI ORIGINAL (com 90 itens): 0.4511239

```

```

cat("NMI PURIFICADO (com", length(itens_sobreviventes), "itens):\n")

```

```

## NMI PURIFICADO (com 37 itens):

```

```

print(comparacao_purificada)

```

```

##                               Method Empirical p.value
## 1 Normalized Mutual Information 0.6995189    0.001

```

4.1 Selecionar itens teoricamente válidos

```

# =====
# 1. IDENTIFICAR E REMOVER OS INTRUSOS
# =====

# Construindo o dataframe que cruza a teoria com o resultado empírico da IA
painel_itens <- data.frame(
  Item = names(gabarito_purificado),
  Dimensao_Teorica = as.character(gabarito_purificado),
  Cluster_EGA = as.numeric(ega_final$wc),
  stringsAsFactors = FALSE
)

painel_filtrado <- painel_itens[painel_itens$Cluster_EGA != 1, ]

itens_perfeitos <- c()

# Loop para analisar os clusters restantes (2, 3 e 4) e limpar intrusos
for (cluster_id in unique(painel_filtrado$Cluster_EGA)) {

  # Pega os itens deste cluster específico
  itens_do_cluster <- painel_filtrado[painel_filtrado$Cluster_EGA == cluster_id, ]

  # Descobre qual é a dimensão teórica dominante (ex: maioria é Ansiedade)
  dimensao_dominante <- names(sort(table(itens_do_cluster$Dimensao_Teorica), decreasing = TRUE)[1])

  # Mantém apenas os itens cuja teoria bate com a dimensão dominante do cluster
  itens_validos <- itens_do_cluster$Item[itens_do_cluster$Dimensao_Teorica == dimensao_dominante]
}

```

```

# Vai guardando os itens perfeitos
itens_perfeitos <- c(itens_perfeitos, itens_validos)
}

cat("Total de itens puros mantidos:", length(itens_perfeitos), "\n")

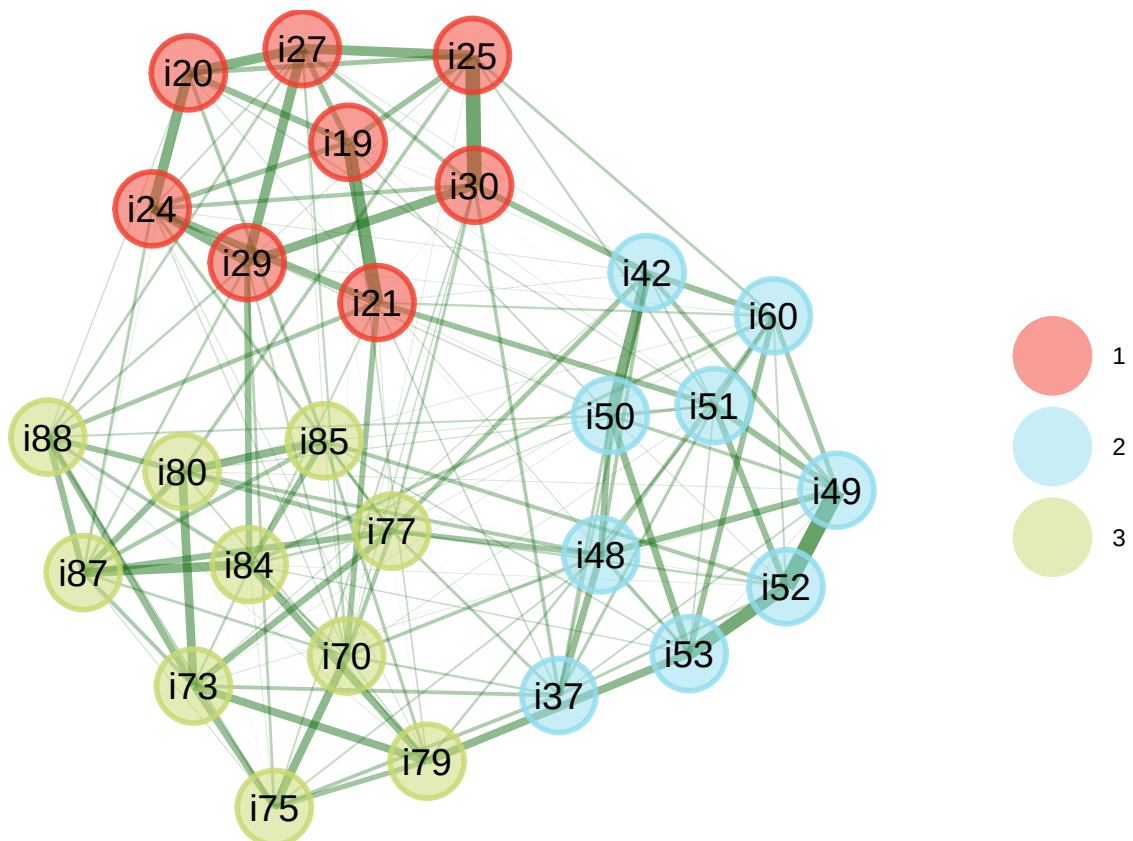
## Total de itens puros mantidos: 27

# Vai imprimir 29 itens (8 de Depressão + 9 de Ansiedade + 12 de Estresse)

# =====
# 2. FILTRAR A MATRIZ E O GABARITO
# =====
matriz_pura <- matriz_final_purificada[, itens_perfeitos]
gabarito_puro <- gabarito_purificado[itens_perfeitos]

# =====
# 3. RODAR EGA FINAL E CALCULAR NMI
# =====
set.seed(123)
ega_puro <- EGA(
  data = matriz_pura,
  model = "glasso",
  plot.EGA = TRUE
)

```



```
# Calcula o NMI da escala 100% limpa
comparacao_pura <- community.compare(
  base = gabarito_puro,
  comparison = ega_puro$wc,
  method = "nmi"
)

cat("\n--- RESULTADO NMI (ESCALA PURA) ---\n")
```

```
##
## --- RESULTADO NMI (ESCALA PURA) ---
```

```
print(comparacao_pura)
```

```
##                                Method Empirical p.value
## 1 Normalized Mutual Information          1    0.001
```

```
cat("\n--- MATRIZ DE CONFUSÃO FINAL ---\n")
```

```
##
## --- MATRIZ DE CONFUSÃO FINAL ---
```

```
print(table(Teoria = gabarito_puro, Cluster_EGA = ega_puro$wc))
```

```
##           Cluster_EGA
## Teoria      1  2  3
## Ansiedade  0  9  0
## Depressao  8  0  0
## Estresse   0  0 10
```

4.2 Salvar matriz de itens estáveis e não redundantes

```
# =====
# CRIAR TABELA COM TEXTOS E EXPORTAR PARA EXCEL (.xlsx)
# =====
library(rio) # Carrega o pacote para exportar
```

```
## Warning: pacote 'rio' foi compilado no R versão 4.5.2
```

```
# 4.1 Monta a tabela básica com as dimensões e clusters
tabela_exportacao <- data.frame(
  ID_Item = names(gabarito_puro),
  Dimensao_Teorica = as.character(gabarito_puro),
  Cluster_EGA = ega_puro$wc[names(gabarito_puro)],
  stringsAsFactors = FALSE
)
```

```
# 4.2 Resgatar os textos originais do banco 'dados'
```

```

# Usa um loop rápido para cruzar o ID com o banco original
textos_resgatados <- sapply(tabela_exportacao$ID_Item, function(id) {

  # Se o UVA juntou o item (ex: i16_i59), pegamos a primeira parte (i16) para achar o texto original
  id_base <- strsplit(id, "_")[[1]][1]

  # Busca a frase correspondente no banco original 'dados'
  # (Certifique-se de que a coluna de textos se chama 'texto' no seu objeto 'dados')
  frase <- dados$texto[dados$item == id_base]

  # Se por acaso não achar, retorna um aviso, senão retorna a frase
  if(length(frase) == 0) return("Texto não encontrado") else return(frase)
})

# 4.3 Adiciona a coluna de texto à nossa tabela de exportação
tabela_exportacao$Texto_do_Item <- textos_resgatados

# 4.4 Reorganiza a ordem das colunas para ficar elegante na planilha
tabela_exportacao <- tabela_exportacao[, c("ID_Item", "Texto_do_Item", "Dimensao_Teorica", "Cluster_EGA")]

# 4.5 Ordena pelas dimensões para a planilha ficar agrupada (Depressão, Ansiedade, Estresse)
tabela_exportacao <- tabela_exportacao[order(tabela_exportacao$Dimensao_Teorica), ]

# 4.6 Exporta para Excel
nome_arquivo <- "itens_escala_final_purificada_com_textos.xlsx"
#export(tabela_exportacao, nome_arquivo)

cat("\n Planilha", nome_arquivo, "salva com sucesso na pasta:\n", getwd(), "\n")

```

```

##
## Planilha itens_escala_final_purificada_com_textos.xlsx salva com sucesso na pasta:
## C:/Users/vinic/OneDrive/Documentos/Psicometria/Generativa

```

```
sessionInfo()
```

```

## R version 4.5.0 (2025-04-11 ucrt)
## Platform: x86_64-w64-mingw32/x64
## Running under: Windows 11 x64 (build 26200)
##
## Matrix products: default
## LAPACK version 3.12.1
##
## locale:
## [1] LC_COLLATE=Portuguese_Brazil.utf8 LC_CTYPE=Portuguese_Brazil.utf8
## [3] LC_MONETARY=Portuguese_Brazil.utf8 LC_NUMERIC=C
## [5] LC_TIME=Portuguese_Brazil.utf8
##
## time zone: America/Sao_Paulo
## tzcode source: internal
##
## attached base packages:
## [1] stats graphics grDevices utils datasets methods base
##

```

```

## other attached packages:
## [1] rio_1.2.4      EGAnet_2.3.0 readxl_1.4.5 embedR_0.1.0
##
## loaded via a namespace (and not attached):
## [1] tidyselect_1.2.1      psych_2.5.3           dplyr_1.1.4
## [4] farver_2.1.2          S7_0.2.1              fastmap_1.2.0
## [7] GGally_2.4.0          digest_0.6.38         rpart_4.1.24
## [10] lifecycle_1.0.5       cluster_2.1.8.1       magrittr_2.0.4
## [13] compiler_4.5.0        rlang_1.1.6           Hmisc_5.2-5
## [16] progress_1.2.3        tools_4.5.0           igraph_2.2.1
## [19] yaml_2.3.10           sna_2.8               data.table_1.17.0
## [22] ggsignif_0.6.4        knitr_1.50            labeling_0.4.3
## [25] prettyunits_1.2.0     htmlwidgets_1.6.4     mnormt_2.1.1
## [28] curl_7.0.0            reticulate_1.44.1     plyr_1.8.9
## [31] RColorBrewer_1.1-3    abind_1.4-8           withr_3.0.2
## [34] foreign_0.8-90        purrr_1.2.1           nnet_7.3-20
## [37] grid_4.5.0            stats4_4.5.0          ggpubr_0.6.2
## [40] lavaan_0.6-20         colorspace_2.1-1      future_1.68.0
## [43] progressr_0.18.0      ggplot2_4.0.2         globals_0.18.0
## [46] scales_1.4.0          gtools_3.9.5          cli_3.6.5
## [49] rmarkdown_2.30        crayon_1.5.3          generics_0.1.4
## [52] rstudioapi_0.17.1     future.apply_1.20.0   httr_1.4.7
## [55] reshape2_1.4.5        pbapply_1.7-4         stringr_1.6.0
## [58] network_1.19.0        parallel_4.5.0        cellranger_1.1.0
## [61] base64enc_0.1-3       vctrs_0.6.5           Matrix_1.7-3
## [64] carData_3.0-5         jsonlite_2.0.0        car_3.1-3
## [67] hms_1.1.4             glasso_1.11           rstatix_0.7.3
## [70] Formula_1.2-5         htmlTable_2.4.3       listenv_0.10.0
## [73] jpeg_0.1-11           tidyr_1.3.1           qgraph_1.9.8
## [76] glue_1.8.0            parallelly_1.44.0     statnet.common_4.11.0
## [79] ggstats_0.11.0        codetools_0.2-20      cowplot_1.2.0
## [82] stringi_1.8.7         gtable_0.3.6          quadprog_1.5-8
## [85] tibble_3.3.1          pillar_1.11.1         htmltools_0.5.8.1
## [88] R6_2.6.1              evaluate_1.0.5        pbivnorm_0.6.0
## [91] lattice_0.22-7        png_0.1-8             backports_1.5.0
## [94] broom_1.0.10          corpcor_1.6.10        Rcpp_1.1.0
## [97] fdrtool_1.2.18        coda_0.19-4.1         gridExtra_2.3
## [100] nlme_3.1-168          checkmate_2.3.2       xfun_0.52
## [103] pkgconfig_2.0.3

```